A01 Understanding Development Environments and Tools for Edge AI and IoT

ITAI 3377 • Group 9
Group Members: Oscar Cortez, Elizabeth Carrillo, Williane Yarro, Luisleonardo Ortiz-Vazquez, & Trevon Woods

January 15, 2026

Edge AI and IoT development usually combines three things: (1) writing and debugging software locally, (2) collecting sensor data and training models, and (3) deploying to constrained edge devices (limited CPU/RAM/power and sometimes unreliable connectivity). This report explains six tools commonly used in that workflow. For each tool, the report includes a description, its purpose, typical use cases, and an illustrative example showing how it could be used in a real project.

a) Visual Studio Code (VS Code)

Description

Visual Studio Code (VS Code) is a lightweight, cross-platform code editor that becomes a full development environment through extensions. It includes features like IntelliSense (smart code completion), built-in debugging, integrated Git/source control, and an integrated terminal so you can code, run, and troubleshoot in one place.

Purpose

- Helps developers write and debug code faster with a single tool that supports many languages.
- Keeps common workflows together (editing, running, debugging, Git, terminal).
- Expands through extensions for embedded development, Python, Node.js, Docker, SSH, linters, and formatters.

Typical Use Cases

- Developing and debugging an edge gateway service (Python or Node.js) that reads sensors and publishes telemetry using MQTT/HTTP.
- Writing embedded firmware (C/C++) for microcontrollers such as ESP32 or Arduino using embedded-focused extensions.

- Working from Windows while targeting Linux devices (Raspberry Pi/Jetson) using WSL or Remote-SSH.

Illustrative Example

A Raspberry Pi acts as a gateway for temperature and vibration sensors. A developer opens the project using VS Code Remote-SSH, runs the gateway service, sets breakpoints in the sensor parsing code, and watches live logs in the integrated terminal. When a sensor packet format changes, the developer inspects variables at a breakpoint, fixes the parser, and redeploys.

b) Node.js

Description

Node.js is an asynchronous, event-driven JavaScript runtime that runs JavaScript outside the browser. Its non-blocking I/O design makes it a strong choice for network-heavy applications, which is common in IoT systems that handle frequent device messages and many connected clients.

Purpose

- Supports server-side scripting for APIs, device gateways, and dashboards using JavaScript/TypeScript.
- Works well for real-time data handling (telemetry streaming, WebSockets, MQTT clients).
- Provides a large ecosystem of packages through npm for fast development.

Typical Use Cases

- Creating an MQTT-to-REST gateway that converts sensor messages into API endpoints for a web/mobile app.
- Hosting a real-time dashboard that streams sensor data to browsers using WebSockets.
- Running edge gateway services that buffer data locally during network outages, then upload later.

Illustrative Example

An industrial vibration sensor publishes MQTT messages several times per second. A Node.js service subscribes to the MQTT topic and forwards live updates to a browser dashboard over WebSockets. Operators can see near real-time changes without needing slow polling requests.

c) Edge Impulse CLI

Description

Edge Impulse CLI is a set of command-line tools used to connect local devices and datasets to Edge Impulse projects. It supports device setup, data ingestion (including from serial-connected boards), and uploading local data files into Edge Impulse for model training.

Purpose

- Makes data collection and dataset uploads repeatable and easier to automate.
- Helps bridge devices that do not have direct internet access by forwarding sensor data through a connected computer.
- Supports creating and managing datasets for Edge AI model development.

Typical Use Cases

- Using a data forwarder to stream live sensor readings over serial into Edge Impulse.
- Uploading existing files (CSV, WAV, etc.) as labeled training data.
- Connecting supported hardware to an Edge Impulse project for testing and deployment workflows.

Illustrative Example

A team wants to detect bearing wear using vibration data. A microcontroller streams accelerometer values over serial. The team uses Edge Impulse CLI to forward the data into an Edge Impulse project and labels samples as “normal” and “worn.” After enough samples are collected, they train a compact model and deploy it back to an edge device for on-device inference.

d) TensorFlow and TensorFlow Lite

Description

TensorFlow is an open-source machine learning platform used to build, train, and evaluate models. TensorFlow Lite (TFLite) is a lightweight deployment format and runtime designed for running models efficiently on edge devices like smartphones, embedded Linux devices, and IoT hardware. A common workflow is training in TensorFlow and then converting the trained model into TensorFlow Lite for edge deployment.

Purpose

- **TensorFlow:** model training, experimentation, evaluation, and exporting models (often using GPUs/TPUs).
- **TensorFlow Lite:** on-device inference with reduced latency, smaller model size, and lower power use. TFLite often uses optimizations like quantization to reduce model size and speed up inference.

Typical Use Cases

- Training an image classifier in TensorFlow and converting it to TFLite for deployment on an edge camera device.
- Running offline keyword spotting or gesture recognition on a wearable or smart device.
- Performing anomaly detection on vibration/temperature data locally on an edge device to reduce bandwidth and cloud dependency.

Illustrative Example

A warehouse camera system needs to detect when a person enters a restricted area. A model is trained in TensorFlow using labeled images. After training, the model is converted to TensorFlow Lite and optimized (for example, using post-training quantization). The edge device runs the TFLite model locally and only sends alerts when a violation is detected, which reduces bandwidth use and improves privacy.

e) Google Colab

Description

Google Colab (Colaboratory) is a hosted Jupyter Notebook environment that runs on Google's cloud infrastructure. It allows users to write and execute Python code through a browser with minimal setup, and it can provide access to GPUs/TPUs depending on availability and runtime settings. Colab notebooks are easy to share, which supports collaboration and reproducibility.

Purpose

- Allows rapid prototyping of machine learning and data analysis without installing heavy software locally.
- Supports collaboration by sharing notebooks and rerunning the same code cells with the same workflow.
- Provides access to cloud compute resources when local hardware is limited.

Typical Use Cases

- Training and evaluating models (e.g., with Keras) and exporting deployment artifacts like TensorFlow Lite models.
- Exploring and cleaning sensor datasets and creating plots before building an embedded pipeline.
- Writing reproducible lab notebooks for course work and team projects.

Illustrative Example

A group trains a small CNN image classifier in Google Colab using a GPU runtime. After evaluating accuracy, they export the final model as a TensorFlow Lite file. They then copy the .tflite model to a Raspberry Pi for benchmarking and on-device testing.

f) Generative AI Coding Tools (GitHub Copilot, OpenAI Codex)

Description

Generative AI coding tools use large language models to suggest, generate, and explain code. GitHub Copilot provides in-editor suggestions and code completion for many languages. OpenAI Codex-style tools can help understand codebases, propose fixes, and assist with debugging and implementation tasks. These tools work best as assistants, where the developer still reviews and verifies the final code.

Purpose

- Speeds up development by generating boilerplate code, examples, and repetitive patterns.
- Helps debugging by suggesting likely fixes and pointing out edge cases.
- Supports learning by explaining unfamiliar libraries, protocols (MQTT/BLE), and coding patterns.

Typical Use Cases

- Generating MQTT client logic including reconnect and resubscribe behavior for IoT gateways.
- Drafting unit tests for preprocessing and feature extraction pipelines.
- Refactoring code to improve performance for edge constraints (memory and latency).

Illustrative Example

An edge gateway occasionally loses Wi-Fi and the MQTT client does not always reconnect cleanly. A developer uses Copilot to suggest reconnection logic (like exponential backoff) and to ensure topics are resubscribed after reconnect. A Codex-style tool can also propose a patch and add tests. The developer reviews the changes, verifies the logic, and then integrates the fix.

References

- Microsoft. Visual Studio Code Documentation. https://code.visualstudio.com/docs
- Node.js. About Node.js and Asynchronous Programming Docs. https://nodejs.org/en/about
- Edge Impulse. Edge Impulse CLI Documentation. https://docs.edgeimpulse.com/tools/clis/edge-impulse-cli
- TensorFlow. TensorFlow Platform and Optimization Guides. https://www.tensorflow.org/
- Google. Colaboratory (Colab) Overview and FAQ. https://colab.google/ ; https://research.google.com/colaboratory/faq.html
- GitHub. GitHub Copilot. https://github.com/features/copilot ; https://docs.github.com/copilot/
- OpenAI. Codex. https://openai.com/codex/ ; https://developers.openai.com/codex/